CS4031 : Compiler Construction

Assignment 03 : Bottom-Up Parser Design & Implementation

SLR(1) and LR(1) Parsing

Ahmad Bilal : 23I-0787, Muhammad Ali : 23I-2565

1. Introduction

Bottom-up parsing is a fundamental technique in compiler design where the parser begins with the input tokens (leaves) and works upward, reducing them step-by-step to the grammar's start symbol. This stands in contrast to top-down parsing, which begins at the start symbol and tries to derive the input.

The two most significant families of bottom-up parsers are SLR(1) (Simple LR) and LR(1) (Canonical LR). Both employ a shift-reduce strategy driven by deterministic finite automata built from sets of LR items, but they differ in the precision of the lookahead information they incorporate.

SLR(1) parsers are constructed from LR(0) items : productions annotated with a dot position indicating how much of the right-hand side has been seen : and use the FOLLOW set of a non-terminal to decide when to reduce. This approach is simple and yields compact tables, but it can introduce spurious reduce actions for grammars whose FOLLOW sets are too broad.

LR(1) parsers extend each LR(0) item with a specific lookahead terminal: [A -> alpha . beta, a]. Reductions are triggered only when the current input symbol matches the item's lookahead, providing finer-grained control that eliminates many conflicts that SLR(1) cannot resolve. The trade-off is a potentially much larger collection of item sets (and hence a larger parsing table).

This assignment implements both parsers in Java, reading a Context-Free Grammar from a file, augmenting it, computing FIRST/FOLLOW sets, building the canonical item-set collections, constructing parsing tables, running the stack-based parsing algorithm on input strings, detecting conflicts, and generating parse trees for accepted strings.

2. Approach

2.1 Data Structures

The implementation is divided into seven Java classes, each with a focused responsibility:

Class	Responsibility
Grammar.java	Stores productions in a LinkedHashMap<String, List<List<String>>>. FIRST and FOLLOW sets are HashMaps of Sets. An inner Production class (lhs + immutable rhs) acts as the canonical token for items and table entries.
Items.java	Houses LR0Item and LR1Item as static inner classes, each holding a Production reference and a dot position (and a lookahead string for LR1). closure0/closure1 and goto0/goto1 implement the standard fixed-point algorithms. Canonical collections are List<Set<LR0Item>> and List<Set<LR1Item>>.
SLRParser.java	Calls buildCanonicalLR0() then iterates states to fill a ParsingTable using FOLLOW sets for reductions.
LR1Parser.java	Calls buildCanonicalLR1() then fills a ParsingTable using item-specific lookaheads for reductions.
ParsingTable.java	ACTION entries stored as List<Action> (a list allows conflict accumulation). GOTO entries stored as Map<Integer, Map<String, Integer>>. Conflict detection is built into setAction().
Stack.java	An ArrayDeque of StackEntry(symbol, state) pairs. popN(n) supports reduce operations. toDisplayString() renders the stack bottom-to-top for trace output.
Tree.java	A parallel node stack (Deque<TreeNode>) mirrors the parser stack. shift() pushes leaf nodes; reduce() pops children, builds a parent, and pushes it back. printNode() renders the tree with Unicode box-drawing characters.

2.2 Grammar Reading and Augmentation

The Grammar.readFromFile() method parses lines of the form:

```
NonTerminal -> production1 | production2 | ...
```

The first non-terminal encountered is taken as the start symbol. augment() prepends a new production "StartSymbolPrime -> StartSymbol" at the front of the productions map, ensuring the augmented start symbol always appears in state I0.

2.3 FIRST and FOLLOW Computation

computeFirstSets() uses a standard fixed-point iteration: for each non-terminal, compute FIRST of each alternative's symbol sequence using firstOfSequence(), which handles epsilon propagation across consecutive symbols. The result converges when no new terminals are added in a pass.

computeFollowSets() seeds FOLLOW(AugmentedStart) = {$} and propagates using the usual rules: for each production A -> alpha B beta, add FIRST(beta) - {epsilon} to FOLLOW(B), and if epsilon in FIRST(beta), also add FOLLOW(A) to FOLLOW(B).

2.4 CLOSURE and GOTO Algorithms

Both closure0() and closure1() use a worklist (LinkedList) to avoid redundant reprocessing. For LR(1) closure, whenever an item [A -> alpha . B beta, a] is processed, the lookahead set for new B-

productions is computed as FIRST(beta + a). Epsilon in that FIRST result contributes the original lookahead a.

goto0/goto1 collect all items whose dot can advance over symbol X, advance the dot, and return the closure of the resulting set. State identity is checked by full set equality (Set.equals()), which is correct because LR0Item and LR1Item override equals() and hashCode() based on production, dotPos, and (for LR1) lookahead.

2.5 Parsing Table Construction

SLRParser.buildTable() iterates over every state and every item. For shift items it calls goto0 and records a SHIFT action. For complete items it looks up FOLLOW(lhs) and records a REDUCE action for each follow terminal. The augmented start's complete item produces ACCEPT on $.

LR1Parser.buildTable() follows the same structure, but instead of FOLLOW(lhs), it uses only item.lookahead for reductions. This finer granularity is the key distinguishing factor.

ParsingTable.setAction() automatically detects conflicts: if an action already exists for a (state, terminal) pair and the new action differs, a descriptive conflict message is appended to the conflicts list.

2.6 Stack-Based Parsing and Parse Tree

Main.parse() implements the standard LR parsing loop. On each iteration it reads the top state s and current input token a, consults ACTION[s, a], and either shifts (pushes symbol+state, advances input), reduces (pops |rhs| entries, pushes lhs + GOTO[newTop, lhs]), or accepts. The Tree object maintains a parallel node stack: shift() pushes leaf nodes, reduce() pops child nodes and creates an internal node.

3. Challenges and Solutions

3.1 Production Identity Across Item Sets

A subtle bug arises if Production objects are compared by reference rather than by value. Because closure0/closure1 create new Production instances on the fly (new Production(B, rhs)), they must override equals() and hashCode(). The implementation compares lhs strings and rhs lists, ensuring that two syntactically identical productions are treated as the same object in sets and maps.

3.2 Lookahead Computation for LR(1) Closure

For an item [A -> alpha . B beta, a], the lookahead for new B-items is FIRST(beta a). Constructing the beta sequence correctly requires slicing the RHS from dotPos+1 onward, then appending the current lookahead. If beta is empty, the lookahead set degenerates to {a}, which is handled naturally by firstOfSequence() treating the single-element list.

3.3 State Equality Check

Checking whether a newly computed GOTO set already exists in the canonical collection requires O(n) iteration over all existing sets. Since Java's LinkedHashSet implements equals() as element-wise comparison, this is correct but potentially slow for large grammars. The worklist-based BFS ensures each new state is added exactly once.

3.4 Conflict Resolution Strategy

When a conflict is detected, the parser stores all conflicting actions in the List<Action> for that cell and resolves to the first entry (shift-preference) when actually parsing. This allows the program to continue building the table and report a full conflict list, rather than aborting on the first conflict.

3.5 Parse Tree Node Ordering

Because the node stack stores children in LIFO order (most recently shifted/reduced is on top), reduce() pops children into a temporary list and reverses it before attaching them to the parent node, preserving the correct left-to-right derivation order in the tree.

4. Test Cases

4.1 Grammar 1 : Simple Arithmetic Expressions

File: grammar1.txt

```
Expr -> Expr + Term | Term
Term -> Factor
Factor -> id
```

Input String	Expected Result	SLR(1)	LR(1)
id	Accept	Accept	Accept
id + id	Accept	Accept	Accept
id + id + id	Accept	Accept	Accept
+ id	Reject	Reject	Reject
id id	Reject	Reject	Reject

4.2 Grammar 2 : Full Arithmetic with Precedence

File: grammar2.txt

```
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id
```

Input String	Expected Result	SLR(1)	LR(1)
id + id * id	Accept	Accept	Accept
(id + id) * id	Accept	Accept	Accept
id * (id + id)	Accept	Accept	Accept

id + * id	Reject	Reject	Reject
(id + id	Reject	Reject	Reject

4.3 Grammar 3 : Classic SLR(1) Conflict Example

File: grammar_with_conflict.txt — This grammar is LR(1) but NOT SLR(1).

```
Start -> L = R | R
L -> * R | id
R -> L
```

SLR(1) result: Shift/Reduce conflict in state 2 on terminal '=': FOLLOW(R) = {$, =} allows a reduce of R -> L in a state that also has a shift on '='. LR(1) resolves this because the item [R -> L •, $] carries only '$' as lookahead, so '=' does not trigger a reduce.

Input String	SLR(1) Result	LR(1) Result
id	Accept (shift-preference)	Accept
id = id	Accept (conflict hidden)	Accept
* id = id	Conflict/Error	Accept
= id	Reject	Reject
* * id = * id	Conflict/Error	Accept

4.4 Grammar 4 : Dangling Else

File: grammar3.txt

```
Stmt -> if Expr then Stmt | if Expr then Stmt else Stmt | other
Expr -> id
```

Input String	SLR(1)	LR(1)
other	Accept	Accept
if id then other	Accept	Accept
if id then other else other	S/R conflict (shift preferred)	S/R conflict
if id then if id then other else other	Conflict : ambiguous	Conflict : ambiguous
if id then	Reject	Reject

Note: The dangling-else grammar is inherently ambiguous and thus not in any LR class. Both parsers report a conflict; the shift preference resolves to the conventional 'else-closest' association.

5. Comparison Analysis

5.1 State Count

Grammar	SLR(1) States	LR(1) States	Ratio

Grammar 1 (simple expr)	7	7	1.0x
Grammar 2 (full arithmetic)	12	22	~1.8x
Grammar 3 (conflict example)	10	15	1.5x
Grammar 4 (dangling else)	~14	~22	~1.6x

LR(1) always generates at least as many states as SLR(1) because each LR(0) state may be split into multiple LR(1) states distinguished by different lookahead sets. For simple grammars the counts may be identical; for complex grammars LR(1) can produce significantly more states.

5.2 Parsing Power

Every SLR(1) grammar is LR(1), but not every LR(1) grammar is SLR(1). The set relationship is:

```
LR(0) ⊂ SLR(1) ⊂ LALR(1) ⊂ LR(1)
```

Grammar 3 above is a concrete demonstration: SLR(1) introduces a spurious shift/reduce conflict because FOLLOW(R) = {$, =}, and '=' appears both as a legitimate shift symbol and (incorrectly) as a reduce trigger. LR(1)'s per-item lookaheads partition the state correctly.

5.3 Table Size and Memory

Parsing table size is proportional to (states) x (terminals + non-terminals). Because LR(1) generates more states, its table is proportionally larger. For Grammar 2, the SLR(1) table has 12 x 9 = 108 cells, while LR(1) has 22 x 9 = 198 cells : approximately 1.8x more memory. For real programming language grammars with hundreds of tokens, LR(1) tables can be prohibitively large, motivating the LALR(1) construction (which merges LR(1) states sharing the same LR(0) core).

5.4 Construction Time

LR(1) construction is slower because (a) there are more states to compute and (b) each closure1() call must compute FIRST sets for lookahead sequences. For the grammars tested here all construction times are sub-second. For industrial grammars, LALR(1) construction (or direct table generation from LR(1) items with state merging) is preferred.

6. Sample Outputs

6.1 Augmented Grammar (Grammar 2)

```
=== Augmented Grammar ===
ExprPrime -> Expr
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id
```

6.2 Selected LR(0) Item Sets (Grammar 2)

```
I0:
```

```
  ExprPrime -> • Expr
  Expr -> • Expr + Term
  Expr -> • Term
  Term -> • Term * Factor
  Term -> • Factor
  Factor -> • ( Expr )
  Factor -> • id

I1:  (GOTO(I0, Expr))
  ExprPrime -> Expr •
  Expr -> Expr • + Term
```

6.3 SLR(1) Parsing Trace : input: id + id * id

Step	Stack	Input	Action
1	0	id + id * id $	Shift 5
2	0 id 5	+ id * id $	Reduce Factor -> id
3	0 Factor 3	+ id * id $	Reduce Term -> Factor
4	0 Term 2	+ id * id $	Reduce Expr -> Term
5	0 Expr 1	+ id * id $	Shift 6
6	0 Expr 1 + 6	id * id $	Shift 5
7	0 Expr 1 + 6 id 5	* id $	Reduce Factor -> id
8	0 Expr 1 + 6 Factor 3	* id $	Reduce Term -> Factor
9	0 Expr 1 + 6 Term 9	* id $	Shift 7
10	0 Expr 1 + 6 Term 9 * 7	id $	Shift 5
11	0 Expr 1 + 6 Term 9 * 7 id 5	$	Reduce Factor -> id
12	0 Expr 1 + 6 Term 9 * 7 Factor 10	$	Reduce Term -> Term * Factor
13	0 Expr 1 + 6 Term 9	$	Reduce Expr -> Expr + Term
14	0 Expr 1	$	Accept

6.4 Parse Tree : id + id * id

```
Expr
├── Expr
│   └── Term
│       └── Factor
│           └── id  [terminal]
├── +  [terminal]
└── Term
    ├── Term
    │   └── Factor
    │       └── id  [terminal]
    ├── *  [terminal]
    └── Factor
        └── id  [terminal]
```

6.5 Conflict Example : Grammar 3

```
*** CONFLICTS DETECTED (SLR(1)) ***
  Shift/Reduce conflict in state 2 on terminal '=': s6 vs r5
=> Grammar is NOT SLR(1)

=> Grammar IS LR(1) (no conflicts)
```

7. Conclusion

This assignment provided hands-on experience with the full pipeline of bottom-up LR parser construction. Key lessons include:

- The distinction between SLR(1) and LR(1) is not merely theoretical : real grammars (such as the classic L-value/R-value grammar) expose situations where FOLLOW sets are too coarse and only per-item lookaheads resolve the ambiguity correctly.
- Correct implementation of equals() and hashCode() on item classes is essential; reference equality silently produces wrong state counts.
- The parallel node-stack approach for parse tree construction is elegant and requires minimal changes to the core parsing loop.
- Conflict detection built into setAction() allows the program to produce a complete conflict report in a single pass over the states, which is far more useful for debugging than aborting on the first conflict.
- The trade-off between SLR(1) and LR(1) (fewer states and faster construction vs. more parsing power) motivates the LALR(1) construction used by most practical parser generators (e.g., yacc, Bison).

Future improvements could include LALR(1) table construction by merging LR(1) states with identical LR(0) cores, a graphical parse tree renderer, and error-recovery mechanisms (e.g., panic-mode or phrase-level recovery) to continue parsing after detecting a syntax error.